Ders 8 Genişletilmiş Notu: İki Boyutlu Faks Kodlaması ve Aritmetik Kodlama

Ders 8 Genişletilmiş Notu: İki Boyutlu Faks Kodlaması ve Aritmetik Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tek Boyutlu Faks Kodlamasının Tekrarı
- 2. İki Boyutlu Kodlamanın Gerekçesi ve Yapısı
- 3. 2D Kodlamada Güvenlik ve k Parametresi
- 4. Coding Line, Reference Line ve Parametreler
- 5. Pass Mode
- 6. Vertical Mode
- 7. Horizontal Mode
- 8. Mod Tablosu, Başlangıç Durumu ve Extension Kodu
- 9. İki Boyutlu Kodlama Örneği
- 10. Faks Sıkıştırması Neden İstatistiksel Bir Yöntemdir?
- 11. Aritmetik Kodlamaya Giriş
- 12. Aralıkların Daraltılması
- 13. Örnek: «a2a2a2a3»
- 14. Kurallar, Ortalama Kod Uzunluğu ve Header
- 15. «SWISS MISS» Örneği: Kodlama
- 16. Kod Çözme
- 17. Sonu Çözülemeyen Girdi ve EOF Sembolü
- 18. Integer Sayılarla Aritmetik Kodlama
- 19. Integer Kod Çözme
- 20. Binary Karşılık ve Huffman ile Karşılaştırma
- 21. PPM'e Giriş: İçerik Tabanlı Modeller
- Örnek Sorular

1. Tek Boyutlu Faks Kodlamasının Tekrarı

Hoca derse, önceki hafta işlenen faks sıkıştırmasını (facsimile compression) hatırlatarak başladı. Faks makinelerinde tek boyutlu ve iki boyutlu olmak üzere iki kodlama vardır ve kullanıcı bunlardan birini seçebilir.

Faks, içinde ne olursa olsun, karşı tarafa gönderdiği belgeyi bir **görüntü** olarak gönderir. Bu görüntü bir resimden farklı olarak piksellerden değil, **pels** (picture element) adı verilen siyah-beyaz noktalardan oluşur. Birçok doküman üzerinde yapılan istatistikler sonucunda her run uzunluğu için bir kod çıkarılmıştır: "bir tane beyaz pels varsa kodu bu, sıfır tane beyaz varsa kodu bu, dört tane arka arkaya siyah pels geliyorsa kodu bu" (ekrandan, 00:20:00).

Her run uzunluğu için ayrı kod çıkarmak yerine kodlar ikiye bölünür:

- **Termination kodları:** 0 ile 63 arasındaki run uzunlukları. Siyah ya da beyaz olmasına göre kodlar değişir.

- **Make-up kodları:** 64 ve 64'ün katları. Belirli bir değerden sonra (slayttaki tabloda 1792'den itibaren) siyahların kodu beyazların koduyla aynıdır. Hoca bunu, bu kadar uzun run'ların çok istisnai olduğu, belki sayfanın dış bordürlerinde görüldüğü ve orada siyah ya da beyaz olmanın çok önemli olmadığı ile açıkladı.

Örneğin taranan satırda arka arkaya 76 beyaz pels varsa 64'ün katı ayrılır: make-up tablosundan 64 beyazın kodu, termination tablosundan 12 beyazın kodu yazılır.

Bu yöntem **bir boyutlu kodlamadır** (Modified Huffman, MH): her satır teker teker kodlanır ve sonuna bir end of line (EOL) kodu konur. Satır sayısı kadar EOL yoksa belgede bir hata vardır ve belgenin yeniden gönderilmesi istenebilir. EOL sayısı doğru olup bir satırın içinde hatalı bitler varsa yalnız o satır hatalı gider, diğerleri doğru gider.

Burada satır, belgede gözle görülen yazı satırı değildir. Bir yazı satırının yüksekliği belki 10 scan line'a karşılık gelir. İnce bir scan line'ın bozuk gitmesi durumunda göz bunu rahatlıkla tolere eder. Belirsizlikleri ortadan kaldırmak için her satırın başına bir beyaz pels konur.

Tek boyutlu kodlamanın başarısı görüntünün içeriğine bağlıdır. Belge metin ve siyah-beyaz görüntülerden oluşuyorsa sıkıştırma oranı yüksektir. Ama gönderilen belge gray scale ya da renkli bir resim olabilir. Faks bunu yine siyah-beyaz gönderir, ama resmin üzerinde işlem yaptığında çok kısa run'lar oluşur. Örneğin arka arkaya 832 beyaz yerine 5 beyaz, 4 siyah, 5 beyaz, 4 siyah gibi run'lar gelirse kod sayısı artar ve sıkıştırma performansı düşer. Performansı yükseltmek için **iki boyutlu kodlama** (two-dimensional coding) kullanılır.

2. İki Boyutlu Kodlamanın Gerekçesi ve Yapısı

Slayttaki bilgiler (ekrandan, 00:27:00):

- Tek boyutlu kodlama gray scale resimlerde iyi sonuç vermediğinden iki boyutlu kodlama geliştirilmiştir.
- Grup 3'lü kodlama faks makinelerinde seçimlik olarak bulunur.
- Her EOL kodundan sonra fazladan bir bit kullanılır. Bu bit 1 ise gelecek satırın 1D, 0 ise 2D ile kodlanacağı anlaşılır.
- 2D kodlama **MMR (Modified Modified READ)** olarak adlandırılır. READ, **Relative Element Address Designate** demektir.
- 2D kodlama, mevcut scan line'ı (**coding line**) bir önünde yer alan satırla (**reference line**) karşılaştırarak aralarındaki farkları kaydederek çalışır.

Hoca adlandırmayı şöyle açıkladı: tek boyutlu kodlama Huffman'ın değiştirilmiş hâliydi (Modified Huffman). İki boyutlu kodlama onu bir kez daha değiştirdiği için adında "modified" iki kez geçer. Kodlama yapılırken iki satır kullanılır: kodlanacak satır (coding line) ve onun üzerinde yer alan, daha önce kodlanmış satır (reference line). Kodlama, bu iki satırın birbiriyle ilişkisine bakılarak oluşturulur.

> **Ders sonrası not:** Büyük olasılıkla ITU standartlarında Grup 3'teki (T.4) 2D kodlamanın adı MR (Modified READ), Grup 4'ün (T.6) kodlamasının adı ise MMR (Modified Modified READ)'dir. Slayt, Grup 3'teki 2D kodlamayı da MMR diye adlandırır.

Farkı kodlamanın sezgisi şudur: dokümanlarda peş peşe gelen satırlar arasında çoğu zaman çok küçük farklar vardır. Elinizde bir A4 dosyası olsun. İlk satır beyaz, arkasındaki satır da beyazsa ikisi birbirinin aynısıdır; aradaki fark alınırsa fark yoktur. O zaman ikinci satır birincinin aynısı gibi kodlanabilir. Ya da üstteki satır bomboş, alttaki satırda yalnız bir siyah pels vardır; aradaki tek fark budur. Alt satırın tamamını kodlamak yerine bu farkı kodlamak daha iyi sıkıştırma verir.

Slayttaki örneğe göre sayfanın başlangıcı yalnız beyaz pels'lerden oluşan bir satır olabilir. Bu satır 1D ile kodlanır. İkinci satırda bir üstte kodlanmış olan satır referans alınır; bu da 2D kodlamadır.

3. 2D Kodlamada Güvenlik ve k Parametresi

İki boyutlu kodlama bir boyutludan daha az güvenilirdir (ekrandan, 00:30:00). Bir hata oluştuğunda hatanın nereden geldiği bilinemez ve hata belgenin sonuna kadar sürer, bütün doküman etkilenir. Bu sebepten belli aralıklarla 2D kodlama kesilir ve bir sonraki satır 1D ile kodlanır.

Soru: Neden belgenin tamamı iki boyutlu kodlanmıyor? **Cevap:** İki boyutlu kodlamada bir hata oluşursa onu tespit etme olasılığı yoktur ve belgenin tamamı hatalı gider. Belirli adımlarda bir boyutlu kodlamaya geçildiğinde hata orada düzelir ve belge düzgün gönderilir.

Ne zaman durulacağı belgenin çözünürlüğüne bağlıdır. Slayttaki kural:

- T4 (Grup 3) standardında bir satır 1D ile kodlandıktan sonra en fazla $k-1$ satır 2D ile kodlanır.
- Standart çözünürlükte $k=2$, yüksek çözünürlükte $k=4$'tür.
- T6 (Grup 4) kodlamada böyle bir zorunluluk yoktur.

Yüksek çözünürlükte $k=4$ ise bir satır 1D kodlanır, ardından üç satır 2D, sonra yine bir satır 1D, sonra üç satır 2D … diye sayfanın sonuna kadar gidilir. Grup 3 seçilirse 1D ve 2D değişimli kullanılmak zorundadır. Grup 4 seçilirse bütün satırlar 2D gönderilebilir; ama bir hata olursa karşı taraf belgeyi olduğu gibi hatalı alır.

1D satırın hatayı neden düzelttiğini hoca şöyle açıkladı: tek boyutlu kodlamada her satırın sonunda bir EOL vardır ve EOL sayısı kaç tane 1D satır kodlandığını gösterir. 2D'de bir bozukluk olduğunda 1D satır durumu düzeltir. Hem EOL'larda hem 2D satırlarda bozukluk varsa belgenin tamamı bozuktur; karşı taraf "gönderdiğin her şey karman çorman, her şeyi yeniden gönder" der.

4. Coding Line, Reference Line ve Parametreler

Hoca bir A4 sayfasını tahtaya çizdi: sayfanın her satırı bir scan line'dır ve scan line'lar birbirine bitişiktir. Her scan line'daki her kare bir pels'tir. İşlem yapılan satıra coding line, onun üstündeki satıra reference line denir. Sayfanın ilk satırında 2D kodlama yapmak imkânsızdır, çünkü üstünde referans alınacak bir satır yoktur; bu satır 1D ile kodlanır.

İki satır karşılaştırılırken beş parametreye bakılır: a_0, a_1, a_2 (coding line'da) ve b_1, b_2 (reference line'da). Hoca bunları pointer gibi düşünmeyi önerdi. Bu beş parametrenin satırlardaki konumlarına göre üç mod vardır:

- Pass mode
- Vertical mode
- Horizontal mode

Slayttaki ifade: 2D kodlamada 3 farklı mod vardır ve 2D kodlama bu 3 mod arasında değişimli kodlama yapar (ekrandan, 00:33:15).

Hoca, örneklerde parametrelerin rastgele bir yerden başlatıldığını belirtti; önemli olan her modun kuralını göstermektir. Pointer'ların genel anlamı şöyledir:

- a_0: coding line'da işlemin kaldığı yer.
- a_1: coding line'da bir sonraki run'ın başlangıcı.
- a_2: a_1'deki run'ın bitişi, yani ondan sonra gelecek run'ın başlangıcı.
- b_1, b_2: reference line'da aynı şekilde, örneğin b_1 bir siyah run'ın başlangıcı, b_2 ondan sonraki beyaz run'ın başlangıcı.

  Hocanın vurgusu: "Bunları ben nasıl aklımda tutarım demeyin. Ben aklımda tutamıyorum, elimde küçük kâğıtlarım var, onlara bakıyorum." Kodunu yazarken de bu kriterlere göre atamalar, karşılaştırmalar ve ötelemeler yapılır. Modların nasıl çalıştığını ezbere bilmeye gerek yoktur; tablolar verilip "bunu çöz" denebilir.

5. Pass Mode

Slayttaki kural (ekrandan, 00:33:15):

- b_2, a_1'in sol tarafındadır.
- a_1 ve b_1 aynı hizada olamaz.
- Bu mod tanımlandığında **Pass Mode (`0001`) + ($b_1 b_2$) uzunluğu** yazılır.
- a_0, b_2'nin altına taşınır; b_1 ve b_2 güncellenir; a_1 ve a_2 değişmez.

Slayttaki şekilde coding line'da a_0'dan sonra 8 beyaz pels vardır ve ardından siyah başlar; a_1 bu siyah run'ın başlangıcıdır. Reference line'da b_1 bir siyah run'ın başlangıcını, b_2 ondan

sonraki beyaz run'ın başlangıcını gösterir. Hoca kontrol etti: b_2, a_1'in sol tarafında mı? Evet. a_1 ile b_1 aynı hizada mı? Hayır. Öyleyse bu pass mode'dur.

Pass mode'un kodu `0001`'dir (hoca "üç tane sıfır ve bir" diye okudu). Arkasına b_1 ile b_2 arasındaki uzunluk yazılır. Şekilde b_1 ile b_2 arasında 2 siyah pels vardır. Bu uzunluğun kodu bir boyutlu kodlamada oluşturulan tablolardan, termination kodlarından alınır: run uzunluğu 2 olan siyah pels'in kodu bulunur ve `0001`'in arkasına yazılır.

Pass mode bittikten sonra pointer'lar yer değiştirir: a_0, b_2'nin altına taşınır. b_2 bir sonraki run'ın başlangıcını gösterir (örneğin ileride bir siyahın başladığı yer). a_1 ve a_2'nin yeri değişmez.

> **Ders sonrası not:** ITU T.4 standardında pass mode kodu büyük olasılıkla yalnız `0001`'dir; arkasından b_1b_2 uzunluğu ayrıca yazılmaz, çünkü alıcı bu uzunluğu kendi elindeki reference line'dan bilir. Slayt ve dersteki örnek ise `0001`'in arkasına uzunluğun kodunu yazar. Burada slayttaki gösterim izlenmiştir.

6. Vertical Mode

Slayttaki kural (ekrandan, 00:40:00): a_1 ile b_1 arasında sağa veya sola doğru belirli bir sayıdan fazla pels olmamalıdır. Yedi farklı durum için 7 farklı kod üretilir:

a_1b_1	Durum	Kod
0	V(0)	`1`
-1	VR(1)	`011`
-2	VR(2)	`000011`
-3	VR(3)	`0000011`
1	VL(1)	`010`
2	VL(2)	`000010`
3	VL(3)	`0000010`

a_1, b_1'in sağında olabilir (VR, "right") ya da solunda olabilir (VL, "left"). a_1 ile b_1 alt alta ise aralarında hiç pels yoktur; değer 0'dır ve kod `1`'dir. a_1, b_1'in sağında ve aralarında bir pels varsa değer -1'dir ve kod `011`'dir; iki pels varsa `000011`'dir.

Hoca "3" değerinin nasıl sayıldığını açıkladı: a_1 ile b_1 arasında aslında iki pels olabilir, ama 1, 2, 3 değerleri bir noktadan diğerinin başına kadar olan mesafeyi gösterir. "Bu aklınızı karıştırıyorsa buna üç de diyebiliriz. Üçten fazla olamaz." Sağa üç, sola üç (altı durum) ve alt alta gelme durumu ile yedi farklı durum vardır.

> **Ders sonrası not:** Slayttaki vertical mode koşulu "2'den fazla pels olmamalıdır" diye yazar. Oysa aynı slayttaki tablo ± 3'e kadar gider ve horizontal mode koşulu "3'ten fazla" olduğundan vertical mode koşulu $|a_1b_1| \leq 3$'tür. Hoca da 00:40:12'de "3'ten fazla olamaz" dedi ve örnekte bu kuralı uyguladı.

Kodlamadan sonra pointer'ların yeri değişir: a_0, a_1'in yerine; a_1, a_2'nin yerine geçer; a_2 bir sonraki run'ın başına gider. b_1, b_2'nin yerine geçer; b_2 bir sonraki run'ın (siyah ya da beyaz, hangisiyse) başına gider. Yeni pointer değerleriyle yeniden pass mode mu, vertical mode mu, horizontal mode mu diye bakılır.

7. Horizontal Mode

Slayttaki kural (ekrandan, 00:42:20):

- a_1 ile b_1 arasında sağa veya sola doğru 3'ten fazla pels olmalıdır.
- Horizontal mode'un kodu `001`'dir ve a_0a_1 ile a_1a_2 arasındaki pels'lerin MH'ye (1D tablosuna) göre kod değerleri gönderilir.
- a_0, a_2'nin yerine geçer; a_1, a_2, b_1 ve b_2 güncellenir.

Slayttaki şekilde a_1, b_1'in sol tarafındadır ve aralarında 4 pels vardır; 4, 3'ten fazla olduğu için bu horizontal mode'dur. Output stream'e `001` yazılır. Arkasından a_0 ile a_1 arasındaki 3 beyazın kodu ve a_1 ile a_2 arasındaki 7 siyahın kodu termination tablosundan alınarak yazılır.

Sonra pointer'lar güncellenir: a_0, a_2'nin yerine geçer. a_1 bir sonraki run nerede başlıyorsa oraya gider, a_2 ondan sonraki run'ı gösterir. b_1, b_2'nin yerine geçer ve b_2 bir sonraki run'ın yerine gider.

8. Mod Tablosu, Başlangıç Durumu ve Extension Kodu

Hoca anlatılanları slayttaki tabloyla özetledi (ekrandan, 00:46:35, "2D Codes for the Group 4 Method"):

Mod	Kodlanan run	Kısaltma	Kod
Pass	b_1b_2	P	`0001` + b_1b_2'nin kodu
Horizontal	a_0a_1, a_1a_2	H	`001` + a_0a_1 ve a_1a_2'nin kodu
Vertical	$a_1b_1 = 0$	V(0)	`1`
	$a_1b_1 = -1, -2, -3$	VR(1), VR(2), VR(3)	`011`, `000011`, `0000011`
	$a_1b_1 = +1, +2, +3$	VL(1), VL(2), VL(3)	`010`, `000010`, `0000010`
Extension			`0000001000`

Örneklerde pointer'lar rastgele yerlere konmuştu; ama bunların ilk başta başladığı bir nokta olmalıdır. Slayttaki başlangıç durumu: tarama işlemi başladığında

- a_0, coding line'daki ilk başlangıç beyaz pels'ini gösterir. Bir boyutlu kodlamada her satırın başına bir beyaz pels konuyordu; işte o pels a_0'ın başlangıcıdır.
- a_1, coding line'daki ilk siyah pels'in başlangıcını gösterir.
- a_2, coding line'da ondan sonra gelen ilk beyaz pels'in başlangıcını gösterir.
- b_1 ve b_2, reference line'daki birinci ve ikinci run'ların başlangıcını gösterir.

a'lar her zaman kodlama yapılan satırın, b'ler bir üst satırın pointer'larıdır. İşleyiş bir döngüdür: mod bulunur, o moda göre işlem yapılıp output'a gönderilir, pointer'ların yeri değişir, yeniden mod bulunur … sayfanın sonuna gelinceye kadar.

Extension kodu (`0000001000`) sayfanın sonuna gelmeden kodlama işlemini iptal etmek veya sayfanın geri kalan kısmını farklı kodla ya da sıkıştırmadan iletmek için kullanılır. Karşı taraf bunu şöyle anlar: gönderen kodlamayı bıraktı; bundan sonra belgenin geri kalanını ya kodlamadan, yani ASCII karşılıklarıyla, sıkıştırmadan gönderecek ya da kodlamayı iptal ediyor. Hoca ayrıca extension kodun belirli sayıda tekrar edilmesinin sayfanın sonuna gelindiğini gösterdiğini söyledi.

9. İki Boyutlu Kodlama Örneği

Hoca, slayttaki "Short example" üzerinden bir satırın 2D kodlanmasını adım adım gösterdi (ekrandan, 00:50:00 ve 01:00:00). Şekilde sayfanın kenarı (edge of page), reference line ve coding line vardır. Pointer'lar satırın başında konmuştur.

Adım 1. Pass mode mu? b_2, a_1'in sol tarafında olmalı; değil. Öyleyse pass mode değildir. Vertical mode mu? a_1 ile b_1 arasında 3'ten fazla pels yok. Öyleyse vertical mode'dur. a_1, b_1'in sol tarafında ve aradaki değer 1 olduğu için bu **VL(1)**'dir; slaytta "$a_1b_1 = 1$, Vertical Mode". Tablodaki kodu `010`'dur.

Pointer'lar vertical mode kuralına göre yer değiştirir: a_0 eski a_1'in yerine (siyahın başına) gelir, a_1 eski a_2'nin yerine geçer, a_2 bir sonraki run'ın başına gider; b_1 eski b_2'nin yerine geçer, b_2 bir sonraki run'ın başına (şu siyahın başına) gelir.

Adım 2. Pass mode mu? Pass mode'da a_1 ve b_1 aynı hizada olamaz; burada aynı hizadadır, öyleyse pass mode değildir. Vertical mode'da sağa ve sola 3'ten fazla pels olamaz; burada ikisi üst üstedir, aralarında 0 pels vardır. Bu **V(0)**'dır ve kodu 1'dir. Pointer'lar yine vertical mode kuralına göre değişir; b_2 bu sefer beyazı gösterir.

Adım 3. Pass mode mu? b_2, a_1'in sol tarafında değil. Vertical mode mu? a_1 ile b_1 arasında 3'ten fazla pels var. Öyleyse bu **horizontal mode**'dur. Önce horizontal mode'un kodu `001`

yazılır. Coding line'da a_0 ile a_1 arasında 3 beyaz, a_1 ile a_2 arasında 4 siyah vardır. Termination tablosundan 3 beyazın kodu `1000` ve 4 siyahın kodu `011` yazılır. Çıkış: `001 1000 011`.

> **Hocanın vurgusu:** "a_1 ile b_1 arasında 7 tane beyaz diyebilirsiniz. Ama hayır. Ben hangi satırla işlem yapıyorum? Coding line ile. Öyleyse coding line'a bakacağım." Yazılan run'lar reference line'dan değil, coding line'dan sayılır.

Pointer'lar horizontal mode kuralına göre değişir: a_0, a_2'nin yerine geçer; a_1, a_2, b_1 ve b_2 güncellenir.

Adım 4. Pass mode kuralı: b_2, a_1'in sol tarafında mı? Evet. Bu **pass mode**'dur. Kod `0001`'dir, arkasına b_1 ile b_2 arasındaki uzunluk yazılır. Hoca coding line'da bu aralıkta 2 beyaz saydı; 2 beyazın termination kodu `0111`'dir. Çıkış: `0001 0111`. Sonra a_0, b_2'nin altına gelir; b_1 ve b_2 güncellenir (b_2 beyazı gösterir); a_1 ve a_2'nin yeri değişmez.

Adım 5. a_1 ile b_1 arasında 3'ten fazla pels yok ve a_1, b_1'in sağ tarafında. Bu yüzden VR'ye bakılır. Aralarında 2 pels vardır; **VR(2)**'nin kodu `000011` yazılır.

Adım 6. Pointer'lar yer değiştirir ve yine vertical mode gelir. Slayttaki çıkışta bu adımın kodu `000010`, yani **VL(2)**'dir.

Adım 7. Sayfanın en sonuna gelindiğinde a_1 ile b_1 arasında 3'ten fazla pels vardır; bu horizontal mode'dur. Hoca burada önce takıldı ("Bu niye 4 white ve 7 black yazıyor?"), sonra pointer'ları yeniden kontrol etti: a_0 beyazı, a_1 siyahı, a_2 sonraki beyazı, b_1 siyahı, b_2 beyazı gösterir. Horizontal mode'un kodu `001` yazılır; a_0 ile a_1 arasında 4 beyaz, a_1 ile a_2 arasında 7 siyah vardır. 4 beyazın kodu `1011`, 7 siyahın kodu `00011`'dir.

Slayttaki satırın tam çıkışı:

```
010 | 1 | 001 1000 011 | 0001 0111 | 000011 | 000010 | 001 1011 00011
VL1   V0  H (3W, 4B)     P (2W)      VR2      VL2      H (4W, 7B)
```

Kodlar slayttaki termination tablosuyla (ekrandan, 00:20:00) karşılaştırıldı ve tuttu: 3 beyaz `1000`, 4 siyah `011`, 2 beyaz `0111`, 4 beyaz `1011`, 7 siyah `00011`.

Satır bitince sonuna EOL konur. Yüksek çözünürlükte gönderiliyorsa ve bu satır 1D kodlanmış bir satırdan sonra geliyorsa sonraki satır da 2D kodlanır: bu sefer şimdiki satır reference line, onun altındaki satır coding line olur. $k-1$ satır 2D kodlandıktan sonra bir satır yalnız 1D kodlanır, sonra yeniden $k-1$ satır 2D … sayfanın sonuna kadar.

Hoca, bunun anlatmasının çok zahmetli olduğunu, ama kodlandıktan sonra "tıkır tıkır" çok hızlı ve iyi bir performansla çalıştığını söyledi.

10. Faks Sıkıştırması Neden İstatistiksel Bir Yöntemdir?

Hoca faks sıkıştırmasını bitirdikten sonra hâlâ istatistiksel yöntemlerin içinde olduklarını vurguladı. Burada kullanılan istatistik, sayfadaki siyah ve beyaz run'ların sayılarıdır: olasılığı çok olanlara daha az bit, az olanlara daha fazla bit atanır. Ama bu bir **statik kodlamadır**: önceden çıkarılmış bir tablo vardır ve işlem onun üzerinde yapılır.

Ardından yeni konuya, aritmetik kodlamaya geçildi. Hoca aradan önce konuyu şöyle tanıttı: şimdiye kadar görülenlerden oldukça farklı bir kodlamadır; karakterlere bir kod vermez, kodlanan dokümanın tamamına bir tane kod verir. On sayfalık bir dokümanın tek bir kod değeri vardır; A'nın, B'nin kod değeri gibi bir şey yoktur.

11. Aritmetik Kodlamaya Giriş

Hoca önce şimdiye kadar görülen istatistiksel yöntemleri hatırlattı (ekrandan, 01:20:00).

Shannon-Fano. Çok basit bir kodlamaydı. Semboller olasılık değerlerine göre sıralanır ve olasılık değerleri yaklaşık olarak tam ortasından ikiye bölünür; bir tarafa 0, bir tarafa 1 verilir. Her aralık yine yaklaşık yarısından bölünerek, artık bölünemeyecek hâle gelene kadar devam edilir. Çok başarılı değildi; başarısı tek bir şarta bağlıydı: olasılık değerlerinin ikinin negatif kuvvetleri cinsinden olması. Gerçek dünyada bir olayın olasılığı hep $1/2$, $1/4$, $1/8$, $1/16$ cinsinden gitmediğine göre iyi sonuç veremiyordu.

Huffman. Huffman da Shannon-Fano gibi ikili ağaç kullanıyordu, ama ağacın dallarını belirli kurallar çerçevesinde ayırıyordu. Ağaç dengeli olmak zorunda değildi ve her seviyede olması gereken bütün kodlar yoktu. Huffman kodları aşağıdan yukarıya oluşturur, ama kodlar yukarıdan aşağıya okunur. Slayta göre Huffman, Shannon-Fano'dan daha iyi sonuç veriyordu.

Bilgi içeriği. Bir sembolün olasılığı P ise o sembolü kodlamak için gereken bit uzunluğu $-\log_2 P$'dir. Slayttaki örnek: bir sembolün olasılığı $0{,}4$ ise bu sembole atanan bit $-\log_2 0{,}4 \approx 1{,}32$'dir; ama pratikte 1 ya da 2 bit verilir.

> **Ek bilgi (derste anlatılmadı):** Huffman her sembole tam sayı uzunluğunda bir kod verdiği için $1{,}32$ bitlik bir bilgi içeriği 1 ya da 2 bitle temsil edilir ve aradaki fark kayıp olur. Aritmetik kodlama bütün girdiye tek bir sayı verdiği için sembol başına kesirli bit harcamaya yaklaşabilir; bu, onun Huffman'dan biraz daha iyi sıkıştırma vermesinin temel sebebidir.

Aritmetik kodlamanın fikri. Slayttaki ifadeler:

- Aritmetik kodlamada tek bir sembole değil, bütün input stream'e bir kod atanır.
- Atanan bu kod $[0, 1)$ aralığında bir sayıdır.
- Kod «9746509» ise bu «0,9746509» olarak yorumlanmalıdır; ancak «0» output stream'e yazılmaz.
- İki pass'te çalışan bir yöntemdir. İlk pass'te sembollerin kullanım sıklıkları çıkarılır, ikinci pass'te sıkıştırma işlemi gerçekleştirilir.

Input stream bir kelime, bir satır, bir sayfa ya da bir kitabın tamamı olabilir. Hoca burada abartmamak gerektiğini söyledi: 500 sayfalık bir kitabın tamamı aritmetik kodlamayla tek sayıya çevrilirse çok küçük bir sayıyla işlem yapmak gerekir ve bu oldukça zor ve zaman alan bir süreç olur.

Bir karakter de olsa, 10.000 karakter de olsa kod 0 ile 1 arasında bir sayıdır. Girdideki karakter sayısı arttıkça, sıfır ile bir arasında sonsuz sayı bulunduğundan, girdiye atanacak sayı o kadar "küçük" (daha çok dijitli, daha dar bir aralıkta) olur. Sayının 0 ile 1 arasında olduğu bilindiği için "0," kısmı yazılmaz ve işlem dijitlerle yapılır.

12. Aralıkların Daraltılması

Birinci pass. Doküman okunur ve sembollerin kullanım sıklıkları, ardından olasılıkları çıkarılır. Hocanın örneğinde üç sembol vardır: a_1'in olasılığı $0{,}4$, a_2'nin $0{,}5$, a_3'ün $0{,}1$.

Semboller olasılıklarına göre $[0, 1)$ aralığına dağıtılır: 0'dan başlanır ve her sembolün olasılığı eklenir.

- a_1: 0 kapalı aralığından başlar, olasılığı $0{,}4$ olduğu için $[0; 0{,}4)$.
- a_2: a_1 $0{,}4$ açık aralığıyla bittiği için $0{,}4$ kapalı aralığıyla başlar; $0{,}4 + 0{,}5 = 0{,}9 \rightarrow [0{,}4; 0{,}9)$.
- a_3: $0{,}9$ kapalı aralığıyla başlar; $0{,}9 + 0{,}1 = 1 \rightarrow [0{,}9; 1)$.

Hoca bunun tersinin de yapılabileceğini söyledi: a_3 0'dan başlatılıp $[0; 0{,}1)$, a_2 $[0{,}1; 0{,}6)$, a_1 $[0{,}6; 1)$ alınabilirdi; sonuçta bir şey değişmez.

İkinci pass. Her yeni sembol mevcut aralığı küçültür. Başlangıçta aralık $[0, 1)$'dir. a_1 geldiğinde aralık biraz küçülür, a_2 geldiğinde biraz daha küçülür, a_3 geldiğinde yine küçülür. Aralığın küçülmesi, atanan sayının dijit sayısının artması demektir. Yeni aralıkların hesaplanmasında kullanılan formüller (ekrandan, 01:30:00 ve 01:40:00):

$$\text{Range} = \text{OldHigh} - \text{OldLow}$$

$$\text{NewHigh} = \text{OldLow} + \text{Range} \times \text{HighRange}(x)$$

$$\text{NewLow} = \text{OldLow} + \text{Range} \times \text{LowRange}(x)$$

LowRange(x) ve HighRange(x), x sembolünün tablodaki alt ve üst limit değerleridir. Örneğin a_1 sembolü gelirse HighRange $= 0{,}4$, LowRange $= 0$ olur. Başlangıçta Low $= 0$, High $= 1$'dir.

> **Hocanın vurgusu:** “Her şey bunun üzerine kurulu.” Önce birinci fazda bütün doküman okunup olasılıklar bulunur, semboller $[0,1)$ aralığına dağıtılır; sonra her yeni gelen sembolle aralık küçültülür ve yeni aralığın alt ve üst değerleri bu formüllerle hesaplanır.

13. Örnek: «a2a2a2a3»

Girdi «a2a2a2a3», aralıklar $a_1 \to [0;0{,}4)$, $a_2 \to [0{,}4;0{,}9)$, $a_3 \to [0{,}9;1)$. Soru: bu girdi aritmetik kodlamayla kodlanırsa output stream’e ne yazılır? (ekrandan, 01:33:40)

Okunan ilk a_2 ile $[0,1)$ aralığı daraltılmaya başlanır.

$$\text{Low} = 0 + (1-0) \times 0{,}4 = 0{,}4 \qquad \text{High} = 0 + (1-0) \times 0{,}9 = 0{,}9$$

İkinci a_2 ile $[0{,}4;0{,}9)$ aralığı daraltılır. Range $= 0{,}9 - 0{,}4 = 0{,}5$.

$$\text{Low} = 0{,}4 + 0{,}5 \times 0{,}4 = 0{,}6 \qquad \text{High} = 0{,}4 + 0{,}5 \times 0{,}9 = 0{,}85$$

Yani girdi yalnız «a2a2» olsaydı output stream’e $0{,}6$ ile $0{,}85$ arasındaki bir sayı yazılacaktı.

Üçüncü a_2 ile $[0{,}6;0{,}85)$ aralığı daraltılır. Range $= 0{,}25$.

$$\text{Low} = 0{,}6 + 0{,}25 \times 0{,}4 = 0{,}7 \qquad \text{High} = 0{,}6 + 0{,}25 \times 0{,}9 = 0{,}825$$

Dördüncü sembol a_3 ile $[0{,}7;0{,}825)$ aralığı daraltılır. Range $= 0{,}125$. a_3’ün alt sınırı $0{,}9$, üst sınırı 1’dir.

$$\text{Low} = 0{,}7 + 0{,}125 \times 0{,}9 = 0{,}8125 \qquad \text{High} = 0{,}7 + 0{,}125 \times 1 = 0{,}825$$

Slaytta High satırında çarpan yanlış yazılmıştı; hoca bunu derste düzeltip “bunun sadece 1 olması lazım” dedi (ekrandan, 01:33:40). Bütün değerler python ile kontrol edildi.

Output stream’e gönderilecek şey $0{,}8125$ ile $0{,}825$ arasındaki bir sayıdır. Sıfır alınmadığı için integer bir sayı gönderilir; bu sayı kaç bitle kodlanıyorsa o kadar yer tutar. Bazıları yalnız low değerini gönderir, bazıları ikisi arasındaki bir değeri. Hoca burada “insanlar böyle şeyleri nasıl düşünüyor” diye hayranlığını dile getirdi.

Bit sayısı. Girdide 4 karakter vardır; her biri 8 bitle 32 bit ederdi. Low değeri olan 8125 yazılırsa: $2^{10} = 1024$, $2^{11} = 2048$, ..., $2^{13} = 8192 > 8125$ olduğu için 8125 sayısı 13 bitle yazılır (python ile kontrol edildi).

> **Ders sonrası not:** Hoca 01:35:14’te bitleri sayarken önce 14 bit dedi, 01:37:07’de “13 bit ile falan kodlanır dedim” diyerek 13 bite döndü. $8125 < 2^{13}$ olduğundan doğrusu 13 bittir.

14. Kurallar, Ortalama Kod Uzunluğu ve Header

Slayttaki kurallar (ekrandan, 01:37:20):

1. $[0,1)$ aralığı ile işleme başlanır.
2. Input stream’de yer alan her «s» sembolü için iki adım tekrarlanır:
   - Geçerli aralık alt aralıklara bölünür; alt aralıkların uzunluğu sembollerin olasılıklarıyla orantılıdır.
   - «s» için alt aralık seçilir ve yeni geçerli aralık olarak tanımlanır.
3. Bütün input stream bu yolla işlendiğinde çıkış, geçerli aralık arasında kalan herhangi bir sayı olacaktır.

Her bir sembolü işlemek için geçerli aralık küçültülür ve bu da kodun daha fazla bitle ifade edilmesine sebep olur. Input stream’in sonuna gelindiğinde elde kalan sayının binary karşılığı, input stream’i kodlamak için gereken bit sayısını verir.

Hocanın vurgusu: Sonuç tek bir sayıdır ve bu sayı tek bir sembolü değil, bütün input stream'i temsil eder.

Ortalama kod uzunluğu:

$$\text{Ort. kod uzunluğu} = \frac{\text{output bit uzunluğu}}{\text{input stream'deki sembol sayısı}}$$

«a2a2a2a3» için output 13 bit, sembol sayısı 4'tür (üç a_2 ve bir a_3): $13/4 = 3{,}25$ bit (python ile kontrol edildi; slaytta hoca "13/" diye not aldı).

Header. Karşı taraf yalnız gönderilen sayıya (örnekte 8125) bakarak orijinal veriyi açamaz. Bunun için başka bilgiler de gönderilmelidir: semboller ve sembollerin sıklık değerleri header'da gönderilir. Karşı taraf header'dan sembolleri ve frekanslarını çekerek aralık tablosunu yeniden oluşturur; bu tablo ve output stream'deki sayıyı kullanarak orijinal girdiyi elde eder.

15. «SWISS MISS» Örneği: Kodlama

Hoca kitaptan alınmış «SWISS MISS» örneğine geçti (ekrandan, 01:40:00). Girdi analiz edilir: 5 tane S, 1 tane W, 2 tane I, 1 tane M ve 1 tane boşluk vardır. Toplam frekans $5+1+2+1+1 = 10$'dur.

Sembol	Frekans	Olasılık	Aralık	CumFreq
S	5	$5/10 = 0{,}5$	$[0{,}5; 1{,}0)$	5
W	1	$1/10 = 0{,}1$	$[0{,}4; 0{,}5)$	4
I	2	$2/10 = 0{,}2$	$[0{,}2; 0{,}4)$	2
M	1	$1/10 = 0{,}1$	$[0{,}1; 0{,}2)$	1
boşluk	1	$1/10 = 0{,}1$	$[0{,}0; 0{,}1)$	0

Toplam CumFreq $= 10$. Slayta göre kümülatif frekans (CumFreq) sütunu kod çözme algoritması tarafından kullanılır.

Kitap tabloyu tersten kurmuştur: 1'den başlayıp olasılıkları çıkararak gider (S $1-0{,}5 = 0{,}5$, W $0{,}5-0{,}1 = 0{,}4$, I $0{,}4-0{,}2 = 0{,}2$, M $0{,}2-0{,}1 = 0{,}1$, boşluk $0{,}1-0{,}1 = 0$). Hoca 0'dan başlayıp olasılıkları ekleyerek kurmayı daha kolay bulduğunu söyledi: boşluk $[0; 0{,}1)$, M $[0{,}1; 0{,}2)$, I $[0{,}2; 0{,}4)$, W $[0{,}4; 0{,}5)$, S $[0{,}5; 1)$. Sonuç aynıdır. Kümülatif değerler de aynı şekilde kurulur: başlangıç 0; boşluk 1 → 1; M 1 → 2; I 2 → 4; W 1 → 5; S 5 → 10.

Kodlama adımları. Low $= 0$, High $= 1$ ile başlanır. Her adımda yeni Low ve High, önceki aralığın low değeri, range'i ve gelen sembolün tablodaki alt ve üst sınırıyla hesaplanır:

Sembol	Low	High
S	$0 + 1 \times 0{,}5 = 0{,}5$	$0 + 1 \times 1 = 1$
W	$0{,}5 + 0{,}5 \times 0{,}4 = 0{,}7$	$0{,}5 + 0{,}5 \times 0{,}5 = 0{,}75$
I	$0{,}7 + 0{,}05 \times 0{,}2 = 0{,}71$	$0{,}7 + 0{,}05 \times 0{,}4 = 0{,}72$
S	$0{,}715$	$0{,}72$
S	$0{,}7175$	$0{,}72$
boşluk	$0{,}7175$	$0{,}71775$
M	$0{,}717525$	$0{,}71755$
I	$0{,}71753$	$0{,}717535$
S	$0{,}7175325$	$0{,}717535$
S	$0{,}71753375$	$0{,}717535$

Tablodaki ara değerler python ile hesaplandı (ara adım eklendi; derste hoca adımları sözlü anlattı ve slayttaki sonuç değerlerini okudu).

Output stream'e yazılacak şey bu aralıktaki bir değerdir. Genelde ya low değeri ya da low ile high arasındaki bir değer alınır. Baştaki sıfır yokmuş gibi alınarak 71753375 sayısı yazılır ve bu

sayı kaç bitle yazılıyorsa «SWISS MISS» girdisi için o kadar bitlik sıkıştırılmış veri gönderilir. Header'da kullanılan semboller ve frekansları vardır; açan taraf tabloyu bunlardan kurar.

16. Kod Çözme

Slayttaki ifadeler (ekrandan, 01:50:00):

- Kod çözücü karşı yönde çalışır.
- Kod çözücü, semboller ve onların aralık değerlerini okuyarak başlangıçta oluşturulmuş olan tabloyu (char, freq, prob, range, cumfreq) tekrar oluşturur.
- İlk dijit olarak 7'yi okur ve bilir ki bu değer 0,7'dir.

Hoca, kod çözücünün yapılan işin tam tersini yaptığını, yani algoritmanın simetrik olduğunu söyledi.

Algoritma. Output stream'deki sayı aslında real bir sayıdır; 0 ile 1 arasında olduğu için başında "0," olduğu bilinir. Her adımda:

1. Sayının tabloda hangi aralığa düştüğüne bakılır; o aralığın sembolü çıkışa yazılır.
2. Sayıdan o sembolün low değeri çıkarılır.
3. Sonuç sembolün range'ine (aralık genişliğine) bölünür. Çıkan sayı bir sonraki adımın sayısıdır.

Sayı 0 olunca işlem biter.

Sembol	Code − low	Range ile bölme
S	$0{,}71753375 - 0{,}5 = 0{,}21753375$	$/0{,}5 = 0{,}4350675$
W	$0{,}4350675 - 0{,}4 = 0{,}0350675$	$/0{,}1 = 0{,}350675$
I	$0{,}350675 - 0{,}2 = 0{,}150675$	$/0{,}2 = 0{,}753375$
S	$0{,}753375 - 0{,}5 = 0{,}253375$	$/0{,}5 = 0{,}50675$
S	$0{,}50675 - 0{,}5 = 0{,}00675$	$/0{,}5 = 0{,}0135$
boşluk	$0{,}0135 - 0 = 0{,}0135$	$/0{,}1 = 0{,}135$
M	$0{,}135 - 0{,}1 = 0{,}035$	$/0{,}1 = 0{,}35$
I	$0{,}35 - 0{,}2 = 0{,}15$	$/0{,}2 = 0{,}75$
S	$0{,}75 - 0{,}5 = 0{,}25$	$/0{,}5 = 0{,}5$
S	$0{,}5 - 0{,}5 = 0$	$/0{,}5 = 0$

Tablo slayttan alındı ve python ile kontrol edildi. İlk adımda $0{,}71753375$, $[0{,}5; 1)$ aralığındadır; bu S'nin aralığıdır, çıkışa S yazılır. $0{,}5$ çıkarılıp $0{,}5$'e bölününce $0{,}4350675$ bulunur. Bu $[0{,}4; 0{,}5)$'e, yani W'ya düşer. W'nun low'u $0{,}4$ çıkarılıp range'i $0{,}1$'e bölünür … Sonunda sonuç 0 olur: "Bitti demektir, elimde artık çözülmesi gereken bir şey yok, başladığım noktaya geri geldim." Çıkışa yazılanlar «SWISS MISS»tir, yani orijinal metin elde edilir.

17. Sonu Çözülemeyen Girdi ve EOF Sembolü

Hoca, bu algoritmada da "kritik noktalar" olduğunu söyleyerek bir problemi gösterdi (ekrandan, 01:55:30).

Örnek. Girdi «a2a2a1a3a3»; üç sembolün aralıkları:

Sembol	Olasılık	Aralık
a_1	$0{,}001838$	$[0{,}998162; 1{,}0)$
a_2	$0{,}975$	$[0{,}023162; 0{,}998162)$
a_3	$0{,}023162$	$[0{,}0; 0{,}023162)$

Slayttaki kodlama adımları (python ile kontrol edildi):

Sembol	Low	High
a_2	0,023162	0,998162
a_2	0,04574495	0,99636995
a_1	0,99462270125	0,99636995
a_3	0,99462270125	0,994663171025547
a_3	0,99462270125	0,994623638610941

a_3'ün aralığı 0'dan başladığı için son iki adımda Low değişmez. Output stream'e ya iki değer arasındaki bir sayı ya da low değeri yazılır; ikisi de tabloya bakıldığında yine aynı aralığı verir.

Çözme. Output stream'e low değeri $0{,}99462270125$ yazılmış olsun.

Sembol	Code − low	Range ile bölme
a_2	$0{,}99462270125 - 0{,}023162 = 0{,}97146070125$	$/0{,}975 = 0{,}99636995$
a_2	$0{,}99636995 - 0{,}023162 = 0{,}97320795$	$/0{,}975 = 0{,}998162$
a_1	$0{,}998162 - 0{,}998162 = 0{,}0$	$/0{,}00138 = 0{,}0$
a_3	$0{,}0 - 0{,}0 = 0{,}0$	$/0{,}023162 = 0{,}0$
a_3	$0{,}0 - 0{,}0 = 0{,}0$	$/0{,}023162 = 0{,}0$

Slaytta ilk satırdaki ara sonuç $0{,}97146170125$ olarak görünür; $0{,}99462270125 - 0{,}023162 = 0{,}97146070125$'tir ve bölüm $0{,}99636995$ ile tutar (python ile kontrol edildi). Üçüncü satırdaki bölen $0{,}00138$ de slaytta tablodaki olasılıktan ($0{,}001838$) farklı yazılmıştır; bölünen 0 olduğu için sonuç değişmez.

a_2, a_2, a_1 çözülür ve a_1'den sonra değer 0 olur. "Sıfır olunca işlem bitiyor demiştik." Ama çözülmesi gereken iki sembol daha vardır. Hoca yanlış bir şey yapılmadığını vurguladı: devam ettirilse a_3, a_3 diye gider, ama algoritmanın durma noktası sıfırdır; ilk sıfırda durur.

Hocanın açıklamasına göre girdinin son karakteri bu tür bir sembolse bu problemle karşılaşılır. Girdi a_3 ile başlasaydı birinci karakter çözülür ve iş biterdi; gerisi yoktur. "Bu acaba kaç kere karşımıza gelir? Ama geldi mi de geliyor."

> **Ders sonrası not:** Hoca 01:56:53'te problemin, girdinin son karakterinin "olasılık değeri sıfır" olmasından kaynaklandığını söyledi. Slayttaki örnekte a_3'ün olasılığı $0{,}023162$'dir, sıfır değildir. Sorunun kaynağı, sembolün **aralığının 0'dan başlamasıdır** (low sınırı 0): böyle bir sembol çözülürken sayı 0 kalır ve algoritma durma koşulunu erken görür.

Çözüm. Slayttaki ifade: ek bir **EOF** sembolü kullanmak gerekir veya kodlayıcı output stream'e önce input stream'in boyutunu yazar ve kontrolü böyle sağlar (ekrandan, 02:00:00). EOF sembolü dokümanda bir kez geçiyormuş gibi tabloya eklenir; olasılığı çok küçüktür ama sıfır değildir. Dokümanın son karakteri artık EOF olur.

Slayttaki EOF örneği:

Sembol	Olasılık	Aralık
eof	$0{,}000001$	$[0{,}999999; 1{,}0)$
a_1	$0{,}001837$	$[0{,}998162; 0{,}999999)$
a_2	$0{,}975$	$[0{,}023162; 0{,}998162)$
a_3	$0{,}023162$	$[0{,}0; 0{,}023162)$

Girdi «a3a3a3a3eof» kodlanır. Her a_3 ile Low 0'da kalır, High küçülür: $0{,}023162$; $0{,}000536478244$; $0{,}00012425909087528$; $0{,}000000287808906285323 5$. EOF gelince aralık $[0{,}000000287808618476417 2; 0{,}0$ olur.

Çözerken: sayı önce a_3'ün aralığına düşer; a_3'ün low'u çıkarılıp range'ine bölünür ve $0{,}0000124258966618912 47$ çıkar. Bu çok küçük olduğu için yine a_3'e düşer … Dördüncü a_3'ten sonra $0{,}999999$ çıkar; bu EOF'un aralığıdır. EOF yazılır, low'u çıkarılıp range'ine bölünür ve sonuç 0 olur. Hepsi çözülmüş ve sıfırda bitirilmiştir.

Soru: Örneği neden değiştirdik? **Cevap:** Hoca, önceki örneğe de EOF eklenip aynı şekilde çözülebileceğini, değişiklik olsun diye ve "daha pis" bir örnek olsun diye bunu seçtiğini söyledi. Bu örnekte aralığı 0'dan başlayan sembol dört kez arka arkaya gelir; önceki örnekte tek bir a_3'te başarısız olunurken EOF eklenince istenen sayıda a_3 gelse bile hiçbir zaman başarısız olunmaz. Dört a_3'te sorun yoksa bir ya da iki a_3'te de olmaz, çünkü girdi her zaman EOF ile biter. Örnek kitaptan alındığı için elle yeniden yazılmadı.

> **Ek bilgi (derste anlatılmadı):** Derste kullanılan «SWISS MISS», «a2a2a1a3a3» ve «Encoding SWISS_MISS by Shifting» örnekleri ile 2D faks kod tablosu ("2D Codes for the Group 4 Method") büyük olasılıkla David Salomon'un *Data Compression: The Complete Reference* kitabından alınmıştır. Hoca da örneklerin "kitaptan" olduğunu söyledi.

18. Integer Sayılarla Aritmetik Kodlama

Şimdiye kadarki işlemler float sayılarla yapıldı. Hoca integer gibi yazsa da işlemlerde başta sıfır varmış gibi float işlemler yapıldığını, bunun işlem süresini uzattığını söyledi. 1 MB'lık bir dosya tek bir sayıyla kodlanır ve bu sayı bazen 500K'lık yer tutan bir float sayı olur. Bu yöntemle yazılmış aritmetik kodlama kodları vardır, ama pratik değildir. Bu yüzden algoritmada değişiklik yapılarak integer sayılarla çalışılır.

Kurallar.

- High ve Low değişkenleri 4 dijit uzunluğundadır.
- Önceki uygulamadaki High ve Low değerlerine bakıldığında en soldaki dijitlerin hiç değişmediği görülür (hep "0,"). Bu bilgi kullanılır: değişmeyen dijitlerle uğraşmak yerine dijitler ötelenerek output stream'e tek bir dijit yazılır. Böylece iki değişkende de kodun tamamı tutulmaz.
- 4 dijitin biri output stream'e atılınca 3 dijit kalır; sağdan bir dijit eklenir. Low değişkenine eklenen dijit 0, High değişkenine eklenen dijit 9'dur.
- Ölçeklendirmede Low ve High değerleri 10.000 ile çarpılır ve High değerinden 1 çıkarılır.

Tablonun kolonları (ekrandan, 02:10:00): 1. kolon okunan input; 2. kolon yeni Low ve High değerleri; 3. kolon bu değerlerin ölçeklendirilmiş hâli; 4. kolon output stream'e gönderilecek dijit; 5. kolon sola ötelemeden sonraki yeni Low ve High değerleri.

Sembol	Low, High (float)	Ölçekli	Dijit	Ötelemeden sonra
S	$0+(1-0)\times 0{,}5 = 0{,}5$; $0+(1-0)\times 1{,}0 = 1{,}0$	5000, 9999	–	5000, 9999
W	$0{,}5 + (1 - 0{,}5) \times 0{,}4 = 0{,}7$; … $\times\, 0{,}5 = 0{,}75$	7000, 7499	7	0000, 4999
I	$0 + (0{,}5 - 0) \times 0{,}2 = 0{,}1$; … $\times\, 0{,}4 = 0{,}2$	1000, 1999	1	0000, 9999
S	$0+(1-0)\times 0{,}5 = 0{,}5$; … $\times\, 1{,}0 = 1{,}0$	5000, 9999	–	5000, 9999
S	$0{,}5 + (1 - 0{,}5) \times 0{,}5 = 0{,}75$; … $\times\, 1{,}0 = 1{,}0$	7500, 9999	–	7500, 9999

Sembol	Low, High (float)	Ölçekli	Dijit	Ötelemeden sonra
boşluk	$0{,}75 + (1 - 0{,}75) \times 0{,}0 = 0{,}75$; ... $\times\, 0{,}1 = 0{,}775$	7500, 7749	7	5000, 7499
M	$0{,}5 + (0{,}75 - 0{,}5) \times 0{,}1 = 0{,}525$; ... $\times\, 0{,}2 = 0{,}55$	5250, 5499	5	2500, 4999
I	$0{,}25 + (0{,}5 - 0{,}25) \times 0{,}2 = 0{,}3$; ... $\times\, 0{,}4 = 0{,}35$	3000, 3499	3	0000, 4999
S	$0 + (0{,}5 - 0) \times 0{,}5 = 0{,}25$; ... $\times\, 1{,}0 = 0{,}5$	2500, 4999	–	2500, 4999
S	$0{,}25 + (0{,}5 - 0{,}25) \times 0{,}5 = 0{,}375$; ... $\times\, 1{,}0 = 0{,}5$	3750, 4999	–	3750, 4999

Output stream'e «717533750» değeri gönderilir (python ile kontrol edildi).

Adımların anlatımı. S geldiğinde Low $= 0$, Range $= 1$; S'nin alt ve üst sınırları $0{,}5$ ve 1'dir. Ölçeklendirince $0{,}5 \times 10.000 = 5000$ ve $1 \times 10.000 - 1 = 9999$ olur. Soldaki dijitler aynı değildir; output stream'e hiçbir şey yazılmaz ve değerler aynen 5. kolona geçer.

W geldiğinde aralık $[0{,}5; 1)$'dir. Low $= 0{,}5$, Range $= 0{,}5$; W'nun sınırları $0{,}4$ ve $0{,}5$'tir. $0{,}7$ ve $0{,}75$ bulunur; ölçekli hâlleri 7000 ve 7499'dur. Sol taraflar aynı (7) olduğu için output stream'e 7 yazılır. Geriye 3 dijit kalır; 4 dijite tamamlamak için Low'a sağdan 0, High'a sağdan 9 eklenir: 0000 ve 4999.

I geldiğinde hoca önce takıldı ve hatayı aradı. Sonra açıkladı: ötelemeden sonraki 0000 ve 4999 aslında 0 ve $0{,}5$ olarak düşünülmelidir. Yani yeni aralık $[0; 0{,}5)$, range $0{,}5$'tir. I'nın sınırları $0{,}2$ ve $0{,}4$ ile $0{,}1$ ve $0{,}2$ bulunur; ölçekli 1000 ve 1999. Soldaki dijitler aynıdır: 1 yazılır, 0000 ve 9999 olur. Burada 9999 aslında 1'dir; High'dan 1 çıkarıldığı için 9999 görünür, 1 eklenince 10.000 olur. Bu yüzden aralık yine $[0; 1)$ kabul edilir.

> **Soru (hocanın kendi sorusu):** Neden aralık $0{,}75$ ile $0{,}9999$ alınmadı? **Cevap:** Ölçeklendirmede High'dan 1 çıkarıldığı için; 1 eklenmiş gibi düşünülerek 9999, 1 olarak kabul edilir. Benzer şekilde 4999 aslında $0{,}5$, 7499 aslında $0{,}75$'tir.

Sonraki iki S'de soldaki dijitler aynı değildir; bir şey yazılmaz. Boşlukta 7500 ve 7749 → 7 yazılır; aralık $[0{,}5; 0{,}75)$ olur. M'de 5250 ve 5499 → 5 yazılır; aralık $[0{,}25; 0{,}5)$. I'da 3000 ve 3499 → 3 yazılır; aralık $[0; 0{,}5)$. Son iki S'de bir şey yazılmaz. Girdi bitince Low'daki 3750 de yazılır. Gönderilen dijitler sırayla 7, 1, 7, 5, 3 ve 3750'dir: «717533750».

Hoca bunun ilk bakışta daha karmaşık görünebileceğini, ama float versiyondaki sayıların büyüklüğüne bakılması gerektiğini söyledi. Float versiyonda doküman uzadıkça sayı küçülür ve dijit sayısı artar. Bu yöntemde ise her zaman 4 dijitlik integer sayılarla işlem yapılır ve yalnız dijitler yazılır.

19. Integer Kod Çözme

Kod çözmede Low $= 0000$ ve High $= 9999$ ile başlanır. Output stream'den her seferinde 4 dijit alınır; ilk 4 dijit 7175'tir. Formüller (ekrandan, 02:20:00 ve 02:23:40):

$$\text{index} = \frac{(\text{Code} - \text{Low} + 1) \times 10 - 1}{\text{High} - \text{Low} + 1}$$

$$\text{Low} = \text{Low} + (\text{High} - \text{Low} + 1) \times \text{LowCumFreq}[X]/10$$

$$\text{High} = \text{Low} + (\text{High} - \text{Low} + 1) \times \text{HighCumFreq}[X]/10 - 1$$

Burada kümülatif frekans kullanılır. Örneğin S'nin low kümülatifi 5, high kümülatifi 10'dur (5 aşağıdan gelir, kendisinin 5'i eklenince 10 olur). Formüldeki 10, toplam kümülatif frekanstır. Hoca bunun standart olmadığını, kitap örneği 10 sembolle yapıldığı için 10 yazıldığını söyledi.

Slayttaki adımlar:

0. Low $= 0000$, High $= 9999$, Code $= 7175$ ile başlanır.
1. index $= [(7175 - 0 + 1) \times 10 - 1]/(9999 - 0 + 1) = 7{,}1759 \to 7$. Bu sayı $0{,}7$ olarak düşünülür ve S'nin $[0{,}5; 1)$ aralığına girer: **S** seçilir. Low $= 0 + 10000 \times 5/10 = 5000$, High $= 0 + 10000 \times 10/10 - 1 = 9999$.
2. index $= [(7175 - 5000 + 1) \times 10 - 1]/(9999 - 5000 + 1) = 4{,}3518 \to 4$: **W** seçilir. Low $= 5000 + 5000 \times 4/10 = 7000$, High $= 5000 + 5000 \times 5/10 - 1 = 7499$. Low ve High'ın ikisinin de sol tarafında 7 olduğu için koddaki 7 artık bitmiştir: 7 ötelenir; Low $= 0000$, High $= 4999$ ve output stream'de sırada bekleyen 3 koda eklenir, Code $= 1753$.
3. index $= [(1753 - 0 + 1) \times 10 - 1]/(4999 - 0 + 1) = 3{,}5078 \to 3$: **I** seçilir. Low $= 0 + 5000 \times 2/10 = 1000$, High $= 0 + 5000 \times 4/10 - 1 = 1999$. 1 ötelenir; Low $= 0000$, High $= 9999$, Code $= 7533$.
4. index $= [(7533 - 0 + 1) \times 10 - 1]/10000 = 7{,}5339 \to 7$: **S** seçilir. Low $= 5000$, High $= 9999$.
5. index $= [(7533 - 5000 + 1) \times 10 - 1]/5000 = 5{,}0678 \to 5$: **S** seçilir. Low $= 5000 + 5000 \times 5/10 = 7500$, High $= 9999$.

Slaytta ilk beş adım gösterildi. Hoca sonraki adımları sözlü olarak hızlıca geçti. Aynı formüllerle python ile hesaplanan geri kalan adımlar (ekranda gösterilmedi):

6. Code $= 7533$, index $= 0{,}1356 \to 0$: **boşluk**. Low $= 7500$, High $= 7749$; 7 ötelenir, Code $= 5337$.
7. index $= 1{,}3516 \to 1$: **M**. 5 ötelenir, Code $= 3375$.
8. index $= 3{,}5036 \to 3$: **I**. 3 ötelenir, Code $= 3750$.
9. index $= 7{,}5018 \to 7$: **S**.
10. index $= 5{,}0036 \to 5$: **S**. Output stream'de alınacak dijit kalmaz.

Çözülen semboller sırayla S, W, I, S, S, boşluk, M, I, S, S'dir: «SWISS MISS».

20. Binary Karşılık ve Huffman ile Karşılaştırma

Slayttaki ifadeler (ekrandan, 02:30:00): pratikte bütün işlemler binary sayılar üzerinde gerçekleşir. «SWISS MISS» stringinin LOW $= 0{,}71753375$, HIGH $= 0{,}717535$'tir. Yaklaşık binary değerleri:

```
L = 0.10110111101100000100101010111
H = 0.10110111101100000101111111011
```

İki sayının binary karşılıkları belirli bir yere kadar aynıdır, sonra ayrılır. Hoca dijitleri saydı ve 29 bit buldu. Slayta göre 10 sembol 29 bitle, yaklaşık 4 byte ile sıkıştırılır. Elde 10 sembol vardı; her biri 8 bitle kodlansaydı 80 bit harcanacaktı. Aritmetik kodlamayla 10 sembol için 29 bit harcanmış olur.

Huffman mı, aritmetik kodlama mı? Yine metnin içeriğine bağlıdır; ama aritmetik kodlama Huffman'dan daha iyi sonuç verir. Aralarında çok büyük bir fark yoktur, ama performansı daha iyidir. Buna karşın Huffman'ı kodlamak ve üzerinde işlem yapmak insanlara daha kolay gelir; bu yüzden Huffman tercih edilir. Aritmetik kodlamanın açık kaynak kodlarında genelde float sayılarla çalışan kodlar görülür; integer versiyonuyla çok az karşılaşılır.

21. PPM'e Giriş: İçerik Tabanlı Modeller

Hoca bir sonraki konu olan PPM'e kısa bir giriş yaptı. PPM, **Prediction with Partial String Matching** diye adlandırıldı.

> **Ders sonrası not:** Algoritmanın kaynaklardaki adı büyük olasılıkla "Prediction by Partial Matching"tir.

Hoca, onlarca sıkıştırma algoritması olduğunu ve derslerde hepsinin anlatılamayacağını, bu dersi farklı üniversitelerde veren hocaların farklı algoritmalara yer verebileceğini söyledi.

> **Hocanın vurgusu:** Herkesin ortak olarak verdiği algoritmalar Huffman, aritmetik kodlama, LZ ve PPM'dir; bunlar mutlaka vardır.

PPM, Cleary ve Witten tarafından ortaya atılmış, Moffat tarafından daha iyi bir hâle getirilmiştir. Bu algoritmayı farklı kılan şey, istatistiksel bir model olması ama **order-N Markov modelini** kullanmasıdır.

Diğer yöntemlerde bir sembolün olasılığı kendisinden önce ya da sonra gelen sembollere bağlı olarak hesaplanmaz; bağımsızdır. Sembol kaç kere geçiyorsa olasılık ona göre hesaplanır, önünde ve arkasında ne olduğu önemli değildir. Oysa bir karakterin sağındaki ve solundaki karakterlerle birlikte kullanımı önemlidir.

Hoca genelde dinamik çalışmayı düşündüğünü söyledi: veri akar ve gelecekte ne geleceği hiçbir zaman bilinmez, ama geçmişte nelerin geldiği bilinir. Bu yüzden Markov modelleri bir sembole olasılık değerini verirken sembolün geçmişindeki karakterlere bakar. $N = 1$ ise geçmişteki bir karaktere, $N = 2$ ise iki karaktere bakılır. "S'nin olasılığı ile AS'nin olasılığı aynı değildir." Bu yüzden bu yöntemlere **içerik (context) tabanlı** sıkıştırma yöntemleri denir.

- Modelde sembole bir olasılık değeri atanmadan önce sembolün içeriği incelenir. Modelin N'i 5 ise sembolün 5 gerisine bakılır; bulunamazsa 4 gerisi, bulunamazsa 3, 2, 1 gerisine bakılır ve durulur.
- Kodlayıcı ve kod çözücü metnin ilerisine erişemediğinden ikisi de geçmişteki metnin içeriğiyle ilgilenir.
- Pratikte bir sembolün içeriği kendisinden önce gelen N semboldür.

Hoca buradaki N'in riskli bir ifade olduğunu açıkladı: bir karakterin olasılığını geçmişindeki 10.000 karaktere bağlı olarak hesapla denirse, ilk 9.999 karakterin önünde 10.000 karakter yoktur ve bunlar olduğu gibi kodlanır. Bu yüzden teoride N çok büyük olabilir, ama pratikte 3'ü 4'ü geçen bir uygulama hocanın görmediği bir şeydir.

21.1 İstatistiksel model ve içerik tabanlı istatistiksel model

Slayttaki **istatistiksel model** (ekrandan, 02:38:05): text'in geçmişinde bulunan her sembolün kullanım sıklığı belirlenir ve bu değere bağlı olarak sembolün olasılığı bulunur. 1217 adet sembolün 34 tanesi q olsun; sembolün olasılığı $34/1217$'dir. Gelen yeni bir q ile olasılık $35/1217$ olarak değişir.

> **Ders sonrası not:** Yeni gelen q ile toplam sembol sayısı da bir artacağı için büyük olasılıkla doğru olasılık $35/1218$'dir. Slayt ve hoca $35/1217$ dedi.

Slayttaki **içerik tabanlı istatistiksel model:** bir S sembolüne, sembolün sıklığına bağlı olmaksızın, context içerisindeki sıklığına bakılarak bir olasılık değeri atanır. Örnek: h'nin olasılığı bağımsız olarak %5, t'nin %30'dur; ama th'nin olasılığı çok daha fazladır.

Hoca bunu ikili kullanımlarla açıkladı. Alfabede üç sembol A, B, C olsun. İkili kombinasyonlar AB, BA, AC, CA, BC, CB gibi bütün olası ikililerdir; bunların metinde olup olmadığı o an bilinmez. Her biri önceden çıkarılır, varsa kullanılır, yoksa kullanılmaz ve olasılık değerleri belirlenir. İngilizce bir dokümanda T, H ve TH aynı olasılığa sahip değildir: H tek başına, T tek başına geçebilir, ama T ve H'nin birlikte geçtiği kelime sayısı fazladır.

21.2 Statik ve adaptif içerik tabanlı yöntemler

Slayttaki **statik içerik tabanlı yöntemler** (ekrandan, 02:40:00):

- Her zaman aynı olasılık değerleri kullanılır. Alfabedeki mümkün olan bütün di-gram ve tri-gram olasılık değerlerinden statik bir tablo oluşturulur.
- Bir S sembolü okunduğunda, kendisinden önce okunmuş C context'ine bağlı olarak tablodan olasılık değerleri çekilerek kullanılır.

Hoca, statik yöntemin önceden alfabedeki bütün ikili, üçlü, istenirse dörtlü, beşli kombinasyonları çıkarıp her biri için olasılık hesapladığını, bir C context'i geldiğinde tabloda o ikiliye nasıl bir olasılık ve kod verildiğine bakıp onu kullandığını söyledi.

Adaptif yöntemde olasılıklar anlık olarak değişir. Hocanın örneğinde içerik «A, B, A, A, A, B, B, A, B» gibi bir dizidir. İlk A geldiğinde A'nın olasılığı 1'dir. B gelince A'nın olasılığı $1/2$, B'nin olasılığı $1/2$ olur. Yeni bir A gelince A'nın olasılığı $2/3$, B'ninki $1/3$ olur. Dinamik Huffman'daki gibi olasılık değerleri durmadan değişir. Yavaş ve karmaşık bir yöntem olmasına rağmen olasılıkları ana göre hesapladığı için sıkıştırma oranı daha yüksektir.

Örnek Sorular

Soru 1 (ekrandan, 01:33:40). Olasılıkları $a_1 = 0{,}4$, $a_2 = 0{,}5$, $a_3 = 0{,}1$ olan semboller için «a2a2a2a3» girdisi aritmetik kodlamayla kodlanırsa output stream'e ne yazılır?

Çözüm (derste verildi): aralıklar $a_1 \to [0; 0{,}4)$, $a_2 \to [0{,}4; 0{,}9)$, $a_3 \to [0{,}9; 1)$. $a_2 \to [0{,}4; 0{,}9)$; $a_2 \to [0{,}6; 0{,}85)$; $a_2 \to [0{,}7; 0{,}825)$; $a_3 \to [0{,}8125; 0{,}825)$. Çıkış bu aralıkta bir sayıdır, örneğin low değeri 8125 (13 bit).

Soru 2 (ekrandan, 01:37:20). Ortalama kod uzunluğu nasıl hesaplanır? «a2a2a2a3» için kaçtır?

Çözüm (derste verildi): output bit uzunluğu / input stream'deki sembol sayısı. 8125 13 bitle yazılır ve 4 sembol vardır: $13/4 = 3{,}25$ bit (bölüm python ile kontrol edildi).

Soru 3 (ekrandan, 01:40:00 ve 01:50:00). «SWISS MISS» girdisi için olasılık tablosunu kurun, girdiyi kodlayın ve elde edilen kodu çözün.

Çözüm (derste verildi): S $[0{,}5; 1)$, W $[0{,}4; 0{,}5)$, I $[0{,}2; 0{,}4)$, M $[0{,}1; 0{,}2)$, boşluk $[0; 0{,}1)$. Kodlama sonunda aralık $[0{,}71753375; 0{,}717535)$'tir. Çözmede $0{,}71753375$'ten başlanır: S ($0{,}4350675$), W ($0{,}350675$), I ($0{,}753375$), S ($0{,}50675$), S ($0{,}0135$), boşluk ($0{,}135$), M ($0{,}35$), I ($0{,}75$), S ($0{,}5$), S (0).

Soru 4 (ekrandan, 01:55:30). Aralıkları $a_1 \to [0{,}998162; 1)$, $a_2 \to [0{,}023162; 0{,}998162)$, $a_3 \to [0; 0{,}023162)$ olan semboller için «a2a2a1a3a3» kodlanıp low değeri gönderilirse çözmede ne olur? Problem nasıl giderilir?

Çözüm (derste verildi): low $= 0{,}99462270125$. Çözmede a_2, a_2, a_1 bulunur ve a_1'den sonra değer 0 olur; algoritma durur ve son iki a_3 çözülmez. Çözüm, olasılığı çok küçük ama sıfır olmayan bir EOF sembolü eklemek (örnekte $[0{,}999999; 1)$) ya da output stream'e önce input stream'in boyutunu yazmaktır.

Soru 5 (ekrandan, 02:10:00). «SWISS MISS» girdisi 4 dijitlik integer Low ve High ile (shifting yöntemiyle) kodlanırsa output stream'e hangi değer gönderilir?

Çözüm (derste verildi): adımlarda sırayla 7 (W), 1 (I), 7 (boşluk), 5 (M), 3 (I) dijitleri ötelenerek yazılır; sonda Low'daki 3750 eklenir: «717533750».

Soru 6 (ekrandan, 02:23:40). «717533750» kodu, Low $= 0000$ ve High $= 9999$ ile başlanarak integer yöntemle nasıl çözülür? İlk adımları gösterin.

Çözüm (derste verildi): Code $= 7175$. index $= 7{,}1759 \to$ S (Low 5000, High 9999); index $= 4{,}3518 \to$ W (Low 7000, High 7499; 7 ötelenir, Low 0000, High 4999, Code 1753); index $= 3{,}5078 \to$ I (1 ötelenir, Code 7533); index $= 7{,}5339 \to$ S; index $= 5{,}0678 \to$ S.

> **Çözüm (derste verilmedi):** Geri kalan adımlar python ile hesaplandı: index $0{,}1356 \to$ boşluk (Code 5337), $1{,}3516 \to$ M (Code 3375), $3{,}5036 \to$ I (Code 3750), $7{,}5018 \to$ S, $5{,}0036 \to$ S. Sonuç «SWISS MISS».

Soru 7 (ekrandan, 00:50:00 ve 01:00:00). Slayttaki "Short example" satırında coding line ile reference line karşılaştırılarak hangi modlar oluşur ve satır nasıl kodlanır?

Çözüm (derste verildi): VL(1) `010`; V(0) `1`; horizontal 3 beyaz + 4 siyah `001 1000 011`; pass + 2 beyaz `0001 0111`; VR(2) `000011`; VL(2) `000010`; horizontal 4 beyaz + 7 siyah `001 1011 00011`. Çıkış: `010 1 001 1000 011 0001 0111 000011 000010 001 1011 00011`, sonra EOL.

Soru 8 (ekrandan, 00:30:00). T4 (Grup 3) standardında yüksek çözünürlükte gönderilen bir belgede satırlar hangi sırayla 1D ve 2D kodlanır? T6'da durum nedir?

Çözüm (derste verildi): yüksek çözünürlükte $k = 4$'tür: bir satır 1D, sonra en fazla $k - 1 = 3$ satır 2D, sonra yine bir satır 1D … (standart çözünürlükte $k = 2$: bir satır 1D, bir satır 2D). T6 (Grup 4) kodlamada böyle bir zorunluluk yoktur; bütün satırlar 2D kodlanabilir.

Dersin Toparlaması

Dersin ilk bölümünde faks sıkıştırması tamamlandı. Tek boyutlu kodlama (MH) gray scale belgelerde kısa run'lar yüzünden başarısız olduğu için iki boyutlu kodlama geliştirilmiştir: kodlanan satır (coding line) bir üstteki satırla (reference line) karşılaştırılır ve farklar üç modla kodlanır. a_0, a_1, a_2, b_1, b_2 pointer'larının konumuna göre pass mode (b_2, a_1'in solunda), vertical mode ($|a_1 b_1| \leq 3$) ya da horizontal mode ($|a_1 b_1| > 3$) seçilir, kodu yazılır ve pointer'lar güncellenir. 2D kodlamada hata tespit edilemediği için Grup 3'te bir satır 1D'den sonra en fazla $k - 1$ satır 2D kodlanır.

İkinci bölümde aritmetik kodlamaya geçildi. Bu yöntem karakterlere değil, bütün girdiye $[0, 1)$ aralığında tek bir sayı atar; her sembol aralığı olasılığıyla orantılı olarak daraltır. Kod çözücü header'daki sembol ve frekanslardan tabloyu kurup sayının düştüğü aralığa göre sembolleri sırayla çıkarır. Aralığı 0'dan başlayan sembollerle biten girdilerde çözme erken durduğu için EOF sembolü eklenir. Float sayıların büyümesi sorunu, 4 dijitlik integer Low ve High ile dijitleri öteleyerek çözülür. Aritmetik kodlama Huffman'dan biraz daha iyi sıkıştırır ama kodlanması daha zordur. Ders, bir sembolün olasılığını önceki N sembole göre veren içerik tabanlı modellere ve PPM algoritmasına giriş yapılarak bitti; hoca bir sonraki derste PPM ve LZW'nin anlatılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.